

Interview Assignment: Agentic Software Engineering System

Objective

Design and implement a working prototype of an agentic system that transforms a software requirement into a reviewable engineering outcome.

The system must demonstrate the ability to:

- Understand requirements
- Decompose work into structured tasks
- Orchestrate multi-step execution
- Generate and validate engineering outputs

The focus is on end-to-end workflow automation across the SDLC, not a generic chatbot.

Scope

The system should support real-world engineering requirements, including:

- Greenfield scenarios: new feature or system development
- Brownfield scenarios: enhancements, refactoring, and bug fixes
- Test and documentation improvements

The system must handle both well-defined and ambiguous requirements.

Core Requirements

1. Requirement Understanding

- Interpret intent and identify ambiguities
- Normalize requirements into a clear engineering problem

2. Task Decomposition

- Break high-level requirements into structured, actionable tasks
- Define dependencies and execution sequence

3. Codebase Reasoning (Brownfield)

- Identify impacted services, modules, APIs, and data flows
- Demonstrate system-level understanding of the existing architecture

4. Workflow Orchestration

- Execute tasks through a coordinated multi-step workflow
- Demonstrate task sequencing and dependency management
- Demonstrate cross-step coordination, not just sequential execution
- Demonstrate error handling and recovery

The solution must show meaningful orchestration across multiple steps, not simple linear task execution.

5. Engineering Output Generation

Produce a cohesive set of:

- Production-quality code or code changes
- API contracts or schema definitions
- Test cases, including unit and integration tests
- Supporting documentation

Outputs must reflect clean design, modularity, and maintainability.

6. Validation and Risk Control

- Identify risks, trade-offs, and failure scenarios
- Define a clear validation approach, including test strategy and verification steps
- Demonstrate guardrails for safe and reliable execution

7. Controlled Autonomy

The system must demonstrate controlled autonomy, where:

- Agents execute tasks independently across multiple steps
- Humans provide oversight, validation, and approval

8. Final Output

Produce a structured engineering summary that includes:

- Implementation plan and rationale
- Generated artifacts

- Risks, trade-offs, and validation approach
- Assumptions and limitations

Deliverables

1. Working Prototype

- A runnable system demonstrating the full workflow
- Accepts a requirement and produces structured engineering output
- Must include the URL shortener use case

2. Architecture Overview

- System components, including agents, tools, and orchestration
- Execution model and control flow
- Key technical and design decisions

3. Example Scenarios

Provide sample inputs and outputs for:

- A greenfield requirement
- A brownfield requirement
- An ambiguous requirement

Each example should demonstrate:

- Task decomposition
- Multi-step orchestration
- Output validation

4. Setup Instructions

- Clear steps to run and evaluate the system

5. Testing Approach

- How correctness and output quality are validated
- Known limitations and trade-offs

Mandatory Use Case

Demonstrate the system with the following requirement:

“Build a scalable URL shortener service with APIs, persistence, and analytics.”

The system must:

- Analyze and decompose the requirement
- Design the architecture
- Generate code, APIs, and tests
- Provide trade-offs and a validation strategy

Evaluation Criteria

Candidates will be assessed on:

- End-to-end workflow completeness
- Strength of system design and architecture
- Depth of task decomposition and orchestration
- Quality and realism of engineering outputs
- Effectiveness of validation and risk management
- Clarity and defensibility of the overall approach

Expectation

This assignment should be approached as production-grade engineering work.

The candidate must demonstrate:

- Strong system design fundamentals
- Ability to orchestrate multi-step workflows across the lifecycle
- Ownership of output quality and validation
- Clear, defensible technical reasoning

The solution should clearly show how requirements are decomposed, executed across steps, and validated under controlled autonomy.